EnergyMonitor Technical Proposal

Juan Esteban Cortés Parra and Miguel Ángel García Artunduaga

Servicio Nacional de Aprendizaje (SENA)

Technologist in Software Analysis and Development (ADSO), group (ficha) 3145556

October 12, 2026 (updated October 28, 2026)

Version Control

Version	Date	Description of change	Authors
1.0	10/12/2026	First version: technology stack, architecture, infrastructure, costs, schedule, risks and discrepancies.	Cortés Parra, J. E.; García Artunduaga, M. A.
1.1	10/20/2026	Alignment with the deployed system: actual production infrastructure (Hostinger KVM 2 VPS for one month, free domain, Nginx, Let's Encrypt), continuous integration and deployment with GitHub Actions, recalculated costs, global administration and new events, status by component, and updated risks and discrepancies.	Cortés Parra, J. E.; García Artunduaga, M. A.
1.2	10/24/2026	Architecture in production: API gateway and security, device and measurement microservices; the core service remains a modular monolith with home, alert, recommendation and notification, whose migration is in progress; communication through internal REST and events on RabbitMQ with a *transactional outbox*; updated stack, infrastructure, status by component and risks.	Cortés Parra, J. E.; García Artunduaga, M. A.
1.3	10/28/2026	Complete migration: homes, notifications, alerts and recommendations as independent services and retirement of the core service (`energy-monitor-backend`); gateway routes, events between services (alerts, recommendations, home activity and account deletion code), account security rules (prior email verification, terms and Google accounts), continuous integration of the eight deployable repositories, tests and status by component as of October 28.	Cortés Parra, J. E.; García Artunduaga, M. A.
1.4	10/28/2026	Web help in Settings (documentation by role, support and news that the build assembles from the git history), photo viewer and photos of the members of a home (`profileImage` field in the internal profile), unusual consumption alert and updated test counts.	Cortés Parra, J. E.; García Artunduaga, M. A.

Abstract

This document presents the technical proposal of EnergyMonitor, a system that lets households see the electrical consumption of each appliance, identify the periods of highest demand, receive alerts and get saving recommendations. It describes the chosen technology stack and its justification, the adopted software architecture (an API gateway and seven independent microservices: security, devices, measurements, homes, notifications, alerts and recommendations), the hosting infrastructure and the project costs. The solution integrates an ESP32 device with a PZEM-004T V3 sensor, services in Java 21 with Spring Boot 4.1.1, a MySQL 8.4 database, a RabbitMQ 4.1 message queue, a Mosquitto message broker with TLS, a React 19 web application and an Android mobile application with Expo SDK 57. The system is deployed at `https://energy-monitor.tech`, on a Hostinger KVM 2 VPS with Docker Compose, and is updated through continuous integration and deployment with GitHub Actions. The estimated total cost is COP 57,021,997, of which COP 8,073,997 is hardware, COP 48,000 is infrastructure (one month of server; the domain was obtained at no cost) and COP 48,900,000 is human resources. The document also includes the schedule, the status by component, the pending technical risks and the differences with the project charter.

Keywords: technical proposal, architecture, microservices, API gateway, costs, infrastructure, continuous integration, Internet of Things

Introduction

EnergyMonitor is a formative project of the Technologist in Software Analysis and Development (ADSO) program at SENA, developed by two trainees and due on October 28, 2026. Its purpose is for households to see how much energy each appliance uses, detect the moments of highest demand, receive alerts for unusual consumption and get saving recommendations based on their own measurements.

This proposal answers the question of whether the project is technically and economically viable with the team's resources. It is organized in three blocks: the justification of the technology stack, the software architecture, and the infrastructure with its costs. It then presents the schedule and the status by component, the technical risks and the differences with the project charter. The functional and non-functional requirements are detailed in the requirements specification (folder `02-especificacion-requisitos`); the objectives, scope and team, in the project charter (folder `01-documento-general-proyecto`); and the deployment procedure, in the implementation and deployment manual (folder `09-implementacion`).

Technology Stack Justification

Solution Overview

Data flows through five pieces. The ESP32 device reads the PZEM-004T V3 sensor over Modbus RTU at 9600 baud (GPIO16 as receive and GPIO17 as transmit, firmware 1.3.8) and publishes over MQTT 3.1.1 with TLS on port 8883 to the Mosquitto broker. The Java services receive the readings: the measurement service stores them in MySQL and announces each one, the alert and recommendation services evaluate them and the notification service delivers them. The web application and the Android mobile application query the services through a REST API, always through the API gateway. The device sends telemetry every 60 seconds and its status every five minutes. Linking is done over Bluetooth Low Energy from the web, with a configuration portal as an alternative. In production, the web app and the API are published at `https://energy-monitor.tech` and devices connect to `mqtt.energy-monitor.tech:8883`.

Decisions by Layer

Table 1 summarizes the technology chosen for each layer and the reason that supports it. The last column compares each choice with common market alternatives. It is a reference comparison, not a record of formal evaluations made during the project.

Table 1

Chosen Technology and Justification by Layer

Layer	Chosen technology	Justification	Market alternatives
Services	Java 21, Spring Boot 4.1.1, Spring Security, Spring Data JPA; Spring Cloud Gateway Server Web MVC (Spring Boot 4.0.8, Spring Cloud 2025.1.3) in the API gateway	Free software tools with efficiency, scalability and ease of integration. Spring Modulith split the original core service into modules with controlled dependencies and made it possible to extract each context into its own service; Spring Cloud Gateway gives the API a single entry point with the same stack.	Node.js with NestJS or Express: broad ecosystem and fast development, but without the verifiable modularity of Spring Modulith. Django (Python) and ASP.NET Core (C#): valid, with less team experience. Gateways such as Kong or NGINX: another technology to maintain.
Messaging between services	RabbitMQ 4.1 with a *transactional outbox*	Free software. The message row is written in the same transaction as the change and a relay publishes it with broker confirmation, so no event is lost when services are separated.	Kafka: more capacity and complexity than a single server needs. Synchronous HTTP calls for everything: they couple the availability of the services.

Layer	Chosen technology	Justification	Market alternatives
Database	MySQL 8.4 with Liquibase migrations	Free software. Liquibase versions database changes with rollback, which gives traceability.	PostgreSQL: equally free and richer in advanced features, unnecessary at this volume. MongoDB: flexible, but the project data is relational (homes, members, devices). Firebase: usage-based cost and provider dependency.
Messaging with the device	MQTT 3.1.1 with TLS on Eclipse Mosquitto	Standard Internet of Things protocol, suitable for small devices (OASIS, 2014). Mosquitto is free and the certificates come from Let's Encrypt. Encryption protects readings in transit.	Periodic HTTP from the device: simpler, but uses more of the microcontroller's resources and has no last will mechanism. CoAP: less support in brokers and tools. Brokers such as EMQX or HiveMQ: more capacity than a single server needs.
Device	ESP32 with PZEM-004T V3 sensor	Low-cost microcontroller with Wi-Fi and Bluetooth (COP 24,000) and an energy sensor of COP 70,000. It records voltage, current, active power and accumulated energy.	Arduino with a Wi-Fi module or Raspberry Pi: the first needs more components and the second has higher cost and consumption. Sensors such as the SCT-013 (current clamp) need calibration and their own circuit, while the PZEM-004T delivers already computed values.
Web application	React 19, Vite 8, Zustand, Axios, React Router, React Hook Form, Zod, i18next, Recharts	Free software. It supports interactive charts, validated forms and an interface in several languages. Web Bluetooth enables linking the device directly from the browser.	Angular or Vue: equally viable, with less prior team knowledge. Next.js: adds server-side rendering, which the application does not need. Create React App: deprecated in favor of Vite.
Mobile application	React Native 0.86 with Expo SDK 57, expo-router, Redux Toolkit	Free software. It reuses the team's React knowledge and serves Android only, which is the defined scope.	Flutter (Dart): good performance, but requires another language and does not reuse what was learned on the web. Native Kotlin: maximum control on Android, at a higher development cost. A progressive web app: less access to phone features.
Infrastructure	Docker Compose on a Hostinger KVM 2 VPS, Nginx with Let's Encrypt certificates and continuous integration and deployment with GitHub Actions	Reproducible deployment on a single low-cost server. The same server hosts the API gateway, the services, the database, the message queue, the broker and the web application. GitHub Actions is already integrated with the team's repositories and allows approving each deployment.	AWS, Google Cloud or Azure: more services and automatic scaling, with variable cost and higher complexity. PaaS platforms: many publish only HTTP and do not accept MQTT port 8883. Other VPS providers (DigitalOcean, Contabo): comparable. Jenkins or GitLab CI: require another server or migrating the repositories.

General Selection Criteria

The decisions in Table 1 share three criteria that guided the selection:

- Zero license cost: all software in the stack is free, and only the server is paid. The domain was obtained at no cost.
- A single server: the expected load of a formative project fits on one VPS. The microservices run on the same server with Docker Compose and share a single MySQL database; the system can grow by increasing the server's resources.
- Team knowledge: the stack uses tools that both members handle, which lowers the risk of delivering on time.

Accepted Limitations

Two decisions have known limits. Web Bluetooth works only in compatible browsers, such as Chrome, and requires a secure context (provided by HTTPS in production); in other browsers the Wi-Fi configuration portal is used. The mobile application is limited to Android, so there is no iPhone version.

Software Architecture

Chosen Architectural Pattern

EnergyMonitor started as a modular monolith built with Spring Modulith: a single deployable application, divided into seven modules with clear boundaries. Between October 24 and 28, 2026 that application was split into independent services: security, devices and measurements on October 24, and homes, notifications, alerts and recommendations on October 28, with the migration of the last four carried out by Miguel Ángel García Artunduaga. Each service has its own repository, container and pipeline, and an API gateway, the single entry point of the API, sits in front of all of them. The core service (`energy-monitor-backend`) was retired. The extraction did not change the public API routes for clients. Kafka is not used. Table 2 describes each component.

Table 2

System Services and Modules

Component	Responsibility
API gateway (`en-monitor-api-gateway`, port 8080)	Single entry point of the API. Answers CORS, validates the token (RS256, issuer and expiration) except on public endpoints, reads the token from the web's HttpOnly cookie, limits public account POSTs (20 per IP and route per minute), rejects `/internal/**` and responds 503 if a service does not answer or 504 if it takes more than 30 s
Security service (`en-monitor-security-service`, port 8082)	Registration with prior email verification and acceptance of the terms, authentication, Google Sign-In, JWT and rotating refresh token, HttpOnly cookie sessions for the web, password recovery, profile, account deletion (with a code sent by email for Google accounts), RBAC and global administration of accounts and roles (`ADMIN` role), support contacts
Device service (`en-monitor-device-service`, port 8084)	Linking, editing, credential reissue and unlinking of devices, appliance catalog, MQTT authentication and authorization, commands to the modules and connectivity (offline after 150 s without data)
Measurement service (`en-monitor-measurement-service`, port 8085)	The only one that stores telemetry; history, latest reading, statistics, home consumption and consumption levels; announces each reading
Home service (`en-monitor-home-service`, port 8086)	Homes, home types, members, OWNER and MEMBER roles, favorites, thresholds, home deletion and ownership transfer; answers the account-deletion and home-membership checks made by the other services
Notification service (`en-monitor-notification-service`, port 8087)	Deliveries, preferences, email (the only one that sends email), Web Push, mobile push, in-app notices and home and device activity
Alert service (`en-monitor-alert-service`, port 8088)	Consumption, connectivity and unusual consumption alerts (a reading of at least 200 W and 2.5 times the usual power of the device in the previous 7 days) from the readings and the device events
Recommendation service (`en-monitor-recommendation-service`, port 8089)	Saving rules, evaluated every hour with the hourly energy from the measurement service

Organization Rules

- Each service owns its domain and its tables, and queries the others only through their internal API or through events; none reads another's tables. `ModularityTests` (Spring Modulith) verified those boundaries while the modules lived in the core service.
- Every service and every module applies hexagonal architecture (`domain`, `application`, `adapter`, `infrastructure`, `api`). The gateway has no domain.
- All services use a single MySQL 8.4 database (`energy_monitor`); each one owns its tables and applies only its own Liquibase changesets, which include rollback. There are no foreign keys between services.
- Identifiers are VARCHAR(10) and deletion is logical through `deleted_at`.

Communication between Modules and Services

Services query each other through internal REST under `/internal/**`, which is accepted only from the Docker network and with the `X-Internal-Token` header, and which the gateway never forwards. Changes are announced with events on RabbitMQ 4.1 through a *transactional outbox*: the message row is written in the same transaction as the change, a relay publishes it every second with broker confirmation, delivery is at least once and the queues are durable, with *dead-letter* after three attempts. All events between services travel on RabbitMQ. Table 3 lists them.

Table 3

Events between Modules and Services

Event	Source	Destination	Channel
`measurement.recorded`	measurement service	alert service	RabbitMQ
`device.connectivity-lost` / `device.connectivity-restored`	device service	alert service	RabbitMQ
`device.linked` / `device.unlinked`	device service	measurement service (cache), alert service and notification service (activity)	RabbitMQ
`alert.raised`	alert service	notification service	RabbitMQ
`recommendation.created`	recommendation service	notification service	RabbitMQ
`account.registered` / `account.administered`	security service	notification service	RabbitMQ
`account.deleted`	security service	home service and notification service (cleanup)	RabbitMQ
`mail.email-verification` / `mail.password-reset` / `mail.account-deletion`	security service	notification service	RabbitMQ
`home.deleted`	home service	device service and notification service	RabbitMQ
`home.created`, `home.member-joined`, `home.member-left`, `home.member-removed`, `home.thresholds-updated`, `home.ownership-transferred`	home service	notification service (activity)	RabbitMQ

The recommendation service does not listen to measurement events: it evaluates recommendations every hour through a scheduler (`RecommendationScheduler`) and asks the measurement service for the hourly energy. A device's loss of connectivity is identified through the status records and the last will and silence mechanisms of the MQTT protocol; the device service announces it with `device.connectivity-lost` and its recovery with `device.connectivity-restored`. On receiving `home.deleted`, the device service unlinks the home's modules and sends them the `unlink` command. Registering an account and the status or role changes made by an administrator produce in-app notices for the other administrators, and the activity of homes and devices reaches users through a single notification queue. Only the notification service sends email: the security service requests the verification, recovery and account-deletion-code emails from it through RabbitMQ.

Interfaces

The API is REST under `/api/v1` and covers authentication, accounts, homes, members, thresholds, devices, measurements, statistics, alerts, recommendations, notification preferences and administration (`/api/v1/admin`: users, roles and notices, with the `USER.READ`, `USER.MANAGE` and `ROLE.MANAGE` permissions). The API gateway distributes the routes: `/api/v1/auth/**`, `/api/v1/admin/users/**`, `/api/v1/admin/roles/**` and `/api/v1/support/**` to the security service; `/api/v1/homes/{homeId}/devices/**` and `/api/v1/appliance-types/**` to the device service; `/api/v1/measurements/**`, `/api/v1/consumption-levels/**` and

`/api/v1/homes/{homeId}/consumption/**` to the measurement service; the rest of `/api/v1/homes/**` and `/api/v1/home-types/**` to the home service; `/api/v1/notifications/**` and `/api/v1/admin/notifications/**` to the notification service; `/api/v1/alerts/**` to the alert service; and `/api/v1/recommendations/**` to the recommendation service. Any other route answers 404. Every service includes Actuator and Swagger (the gateway, only Actuator); in production Swagger does not go through the proxy. There are also internal MQTT authentication and authorization endpoints for Mosquitto in the device service, with a five-minute cache, which are not exposed to the Internet.

Security

- JWT tokens signed with RS256. The access token lasts 15 minutes.
- Opaque, stateful refresh tokens with rotation and reuse detection, stored in the database.
- Global permissions queried dynamically, with no roles inside the JWT; the other services request them from the security service through its internal API, with a short cache.
- The web keeps the session in HttpOnly cookies set by the security service; the gateway reads the token from that cookie.
- Identity is taken from the `sub` claim through CurrentUserResolver. The X-User-Id header is ignored.
- CORS closed by default, with origins defined through environment variables; it is answered by the API gateway. In production the web app and the API share one origin.
- Google Sign-In requires configured OAuth credentials. An account created with Google has no password: it cannot change it and confirms its deletion with a code sent to its email.
- No account gets in without accepting the terms and conditions (428 if missing); registration with a password creates a 24-hour pending request and the account exists only once the email is verified, whose domain is checked (providers with a typing error and nonexistent domains are rejected).
- Devices authenticate to the broker with `device_code` and `api_key`; the device service stores only the SHA-256 hash of the key.
- Internal calls between services require the Docker network and the `X-Internal-Token` header, compared in constant time.
- In production: HTTPS and MQTT with Let's Encrypt certificates, `/internal/**` routes not publicly exposed, and `X-Forwarded-For` replaced with the real IP at the proxy.
- The protection of users' personal data refers to Law 1581 of 2012 (Congreso de la República de Colombia, 2012).

Relevant Domain Rules

Global roles are managed by the security service, while the OWNER and MEMBER roles are per home, so a user can belong to several homes with different roles. The global `ADMIN` role grants access to no home, and an administrator cannot change their own status or roles. Thresholds belong to the home, with default values of 6 kWh daily and 180 kWh monthly. The power levels are LOW (0 to 500 W), MEDIUM (500 to 1500 W), HIGH (1500 to 3000 W) and CRITICAL (from 3000 W). Account deletion is transactional: it validates the password, closes the sessions, logically deletes personal data, releases the email and manages memberships.

Clients

The web application separates its features by characteristic, manages texts with i18n and concentrates HTTP calls in `src/services/`. It is tested with Vitest and Testing Library. An administrator account sees only the administration experience (users, notifications and settings). Settings offers the help in modals: documentation according to the role, technical support (only for users, who create the ticket as an email to an administrator) and news, which the build assembles from the `feat`, `fix` and `perf` commits of the git history, so every web deployment updates them. The profile photo is enlarged in a viewer inside the page (the CSP does not allow requesting `data:` with `fetch`) and the members of a home show their photos: the internal profile of security

includes `profileImage` and the home service returns it. The mobile application uses the expo-router structure and per-component style files, and is available in Spanish, English, French and Portuguese. On the Android emulator 10.0.2.2 is used instead of localhost, and CORS does not apply in React Native.

Development Conventions

Code is written in English and academic deliverables in Spanish. Commits follow Conventional Commits with a module scope and an imperative English message. Branches are `feat/*`, `fix/*` or `chore/*`, with integration through a pull request to `dev` and then to `main` in every repository. Every pull request must pass its automated tests in GitHub Actions before being merged.

Infrastructure and Costs

Hosting Infrastructure

Table 4 describes the production infrastructure.

Table 4

Hosting Infrastructure

Element	Detail
Provider	Hostinger
Plan	VPS KVM 2: 2 vCPU, 8 GB of RAM and 100 GB NVMe, with Ubuntu 26.04 LTS
Contract	One month, for COP 48,000. On a yearly contract the plan costs COP 27,900 per month
Deployment	Docker Compose from the `energy-monitor-infra` repository, with the API gateway, the seven services (security, devices, measurements, homes, notifications, alerts and recommendations), MySQL 8.4, RabbitMQ 4.1, Mosquitto and the web application
Access	Nginx on the server as the only HTTPS entry point; only ports 80, 443 and 8883 are published
Certificates	Let's Encrypt, free of charge and renewed automatically
Domain	`energy-monitor.tech` (web and API) and `mqtt.energy-monitor.tech` (broker), obtained at no cost
Backups	MySQL copy every day and before every deployment, kept for 14 days on the server

Continuous Integration and Deployment

The repositories of the API gateway, the seven services, the web application and the infrastructure have GitHub Actions pipelines (the notification service added its own on October 28, 2026); the mobile application has no pipeline. Every pull request runs the tests (as of October 28, 2026: 774 for the security service, 260 for the home service, 235 for the notification service, 151 for the measurement service, 129 for the device service, 128 for the alert service, 71 for the recommendation service and 69 for the gateway, against MySQL 8.4 for those that need it; the 253 web tests with its build; and validation of the infrastructure configuration), which are required to merge. Every change that reaches `main` is deployed to the server after a manual approval of the `production` environment. The deployment takes a database backup, rebuilds only the changed component, checks the health of all the services and, if it fails, goes back to the previous version. The details are in the implementation and deployment manual (Cortés Parra & García Artunduaga, 2026).

Hardware Cost

Table 5 details the hardware. The first ten lines are the prototype of the measuring device, and the last two are the development team's equipment.

Table 5

Hardware Cost (COP)

Item	Quantity	Unit cost	Subtotal
ESP32 with Wi-Fi and Bluetooth	1	24,000	24,000
PZEM-004T-100A V3 meter	1	70,000	70,000
HLK-PM03 AC-DC power supply	1	20,000	20,000
20 cm male-to-male jumper wires	40	300	12,000
20 cm male-to-female jumper wires	40	300	12,000
2x14 AWG sheathed cable	3	3,333	9,999
Gauge 14 red and black wire	1	2,400	2,400
JST XH2.54 connector	1	1,300	1,300
20 cm female-to-female jumper wires	6	383	2,298
Plastic box (prototype casing)	1	20,000	20,000
Development team computer	2	3,500,000	7,000,000
Development team phone	2	450,000	900,000
Total hardware			**8,073,997**

Of this total, COP 173,997 is the prototype of the measuring device. It is the direct spending on components, close to the COP 180,000 stated in the project charter.

Software, Licensing and Infrastructure Cost

All software in the stack is free and generates no license cost. Table 6 separates what is free from what is paid.

Table 6

Software, Licensing and Infrastructure Cost (COP)

Item	Type	Cost
Java 21 (OpenJDK), Spring Boot, Spring Modulith, Spring Security, Spring Data JPA, Spring Cloud Gateway	Free	0
RabbitMQ	Free	0
MySQL 8.4 and Liquibase	Free	0
Eclipse Mosquitto	Free	0
Docker, Docker Compose and Nginx	Free	0
React 19, Vite 8, Zustand	Free	0
Expo SDK 57 and React Native 0.86	Free	0
Arduino IDE and Visual Studio Code	Free	0
Let's Encrypt (TLS certificates)	Free	0
GitHub and GitHub Actions	Free of charge	0
`energy-monitor.tech` domain	Free of charge	0
Hostinger VPS KVM 2, one-month contract	Paid	48,000
Total software and infrastructure		**48,000**

The plan's price depends on the contract term. The project contracted it for one month, enough for the demonstration and the final presentation. For continuous operation the yearly contract is better: COP 27,900 per month, equal to COP 334,800 per year, against COP 576,000 if it is renewed month by month. The free domain must be reviewed before it expires, because its renewal may have a cost.

Third-Party Services

Table 7 lists the external services the system uses. At the project's current volume none of them has a cost.

Table 7

Third-Party Services

Service	Use	Cost
Let's Encrypt	TLS certificates	0
Google Sign-In	Google sign-in on the web	0
Gmail (SMTP with an app password)	Sending verification, recovery and notification messages	0
Web Push (VAPID)	Browser notifications	0
Expo Push	Mobile application notifications	0
GitHub Actions	Tests and continuous deployment	0

Human Resources Cost

Table 8 values the team's work over seven months. It is an estimated cost to size the project, not an actual payment.

Table 8

Estimated Human Resources Cost (COP)

Role	Months	Monthly cost	Subtotal
Project lead	7	2,000,000	14,000,000
Software developer	7	2,500,000	17,500,000
Database manager	4	1,200,000	4,800,000
Analyst	7	1,800,000	12,600,000
Total human resources			**48,900,000**

Consolidated Budget

Table 9

Consolidated Project Budget (COP)

Category	Subtotal	Share
Hardware	8,073,997	14.16%
Software and infrastructure	48,000	0.08%
Human resources (7 months)	48,900,000	85.76%
Other	0	0.00%
Estimated total	**57,021,997**	**100%**

Nearly 86% of the cost is the team's work. Excluding human resources, the investment in equipment, prototype and server is COP 8,121,997, and the part that actually has to be paid on a recurring basis is the server.

Costs for Users

Users pay no software licenses. They need an Android phone or a modern browser. Table 10 shows reference price ranges for the equipment.

Table 10

Reference Price Ranges of User Equipment (COP)

Equipment	Price range
Xiaomi Redmi 13 phone	450,000 to 700,000

Equipment	Price range
Samsung Galaxy A55 phone	1,600,000 to 2,200,000
HP 240 G9 laptop (Core i5, 8 GB, 512 GB SSD)	2,500,000 to 2,900,000
Dell Inspiron 15 laptop (Core i5, 8 GB)	1,800,000 to 2,500,000

The cost of the measuring device for one home is that of its prototype, about COP 174,000 in components. A sale price or serial production cost is outside the scope.

Maintenance and Licensing

The maintenance plan has three modes. Corrective maintenance handles communication, loading, notification and history failures; a failed deployment is rolled back automatically. Preventive maintenance includes updating dependencies (Spring Boot, React, Expo, firmware), the automatic MySQL backups (daily and before every deployment), automatic certificate renewal and security tests. Evolutionary maintenance considers smart meters, over-the-air firmware updates (today done by USB cable), expanding the administration interface (support and content management) and adaptation to new versions of Android and browsers. Corrective support is provided during the first month after deployment.

The software and hardware are licensed under a free-use modality with community recognition: they may be used, copied and modified for educational, research and social purposes, keeping the reference to the team and SENA. Commercial use requires a formal agreement between the developers and SENA, and may carry a license fee.

Schedule and Current Status

Schedule

The project started on May 1, 2026 and is delivered on October 28, 2026, the date set for the evaluation; the go-live took place between October 16 and 20, before the delivery, and the services architecture reached production on October 24 (security, devices and measurements) and October 28 (homes, notifications, alerts and recommendations). Table 11 shows the phases.

Table 11

Project Phases and Dates

Phase	Period	Expected result
1. Foundation and core loop	May 1 to June 30	A user registers, creates a home, links a device and sees measurements arrive
2. Consumption visibility and alerts	July 1 to August 31	Consumption is visible and the system warns when something is abnormal
3. Recommendations and internationalization	September 1 to 30	Recommendations based on own consumption and an interface in more than one language
4. Consolidation	October 1 to 16	System ready for demonstration and defense
5. Go-live and delivery	October 16 to 28	System published at `https://energy-monitor.tech` with continuous integration and deployment (October 16 to 20), API gateway and security, device and measurement microservices in production (October 24), complete migration of the other modules and retirement of the core service (October 28) and delivery with its defense on October 28

Status by Component

Table 12 presents progress as of October 28, 2026.

Table 12

Status by Component as of October 28, 2026

Component	Status
Measuring device (firmware 1.3.8)	Operational. Linking a meter in production is pending evidence
API gateway	Deployed in production; 69 automated tests on every pull request
Security service	Deployed in production; 774 automated tests
Device service	Deployed in production; 129 automated tests
Measurement service	Deployed in production; 151 automated tests
Home service	Deployed in production on October 28, 2026; 260 automated tests
Notification service	Deployed in production on October 28, 2026; 235 automated tests
Alert service	Deployed in production on October 28, 2026; 128 automated tests
Recommendation service	Deployed in production on October 28, 2026; 71 automated tests
Web application	Deployed in production at `https://energy-monitor.tech`; 253 automated tests
Android mobile application	Under development, integrated with the API; 78 unit tests in 8 suites; outside the production deployment
Infrastructure	Hostinger KVM 2 VPS with Docker Compose, Nginx, Let's Encrypt, RabbitMQ and GitHub Actions; end-to-end tests of the services on the local stack (`device-flow` 24, `measurement-flow` 28, `alert-flow` 36, `activity-flow` 60 and `account-deletion-flow` 12, all green)

Pending is device history management by linking period. The migration to microservices was completed on October 28, 2026. In production, the functional evidence listed in the implementation and deployment manual is still missing: linked meter, email, administration, scheduled backup and certificate renewal.

Pending Technical Risks

Table 13

Pending Technical Risks

Code	Risk	Effect
RT-01	Mitigated: the device `api_key` is no longer stored in clear; only its SHA-256 hash is kept (`api_key_hash`, changesets `device-007` and `device-008`)	Keys of already linked devices are renewed when they are linked again
RT-02	Telemetry uses MQTT with QoS 0	Possible loss of readings. QoS 1 will be evaluated
RT-03	Mitigated: the web no longer stores the tokens in localStorage; the session travels in HttpOnly cookies set by the security service, which no script can read	Risk of theft through XSS reduced
RT-04	No over-the-air (OTA) firmware update	The device is updated only by cable
RT-05	Authentication attempt limits are kept in memory	They reset when the service restarts
RT-06	Partially mitigated: persistence tests require an active database, but in GitHub Actions they run against a MySQL 8.4 service container	Locally a MySQL of one's own is still needed

Code	Risk	Effect
RT-07	The access token remains valid for up to 15 minutes after account deletion	Brief window of residual access
RT-08	Some edge cases of connectivity alerts do not resolve automatically	Alerts that need manual review
RT-09	Web Bluetooth has compatibility limitations and requires a secure context	The configuration portal must be used in other browsers; in production HTTPS provides the secure context
RT-10	MySQL backups are stored on the same server as the database	A server failure loses the data and its backups at the same time
RT-11	There is no availability monitoring or alerting in production	An outage is only detected during a deployment or when a user reports it

Documentary Discrepancies

Table 14 points out the differences between what other project documents say and what is implemented or broken down in this proposal. They were verified against the code of the core service and the web application. This proposal always adopts the implemented state.

Table 14

Detected Differences and Proposed Action

Code	Source of the difference	What it says	Verified state	Action
DD-01	Web application README	Recommendations are simulated because the `recommendation` module of the core service does not exist	The recommendation service (formerly the `recommendation` module of the core service) exists and evaluates five saving rules	Correct the web application README. Still open as of October 20, 2026
DD-02	Project charter	It said: investment of about COP 180,000 with no total cost	Total estimated budget of COP 57,021,997. The COP 180,000 is the direct spending on prototype components (COP 173,997)	Resolved: version 1.1 of the charter includes the total cost
DD-03	Project charter	It said: hosting and paid services not contemplated	Paid Hostinger KVM 2 VPS (COP 48,000 for one month) and free domain	Resolved: version 1.1 of the charter reflects the paid infrastructure

References

Congreso de la República de Colombia. (2012, October 17). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales* [Law 1581 of 2012: By which general provisions for the protection of personal data are issued]. Diario Oficial No. 48,587.

Cortés Parra, J. E., & García Artunduaga, M. A. (2026). *EnergyMonitor implementation and deployment manual*. SENA – CIES Regional Huila.

OASIS. (2014). *MQTT Version 3.1.1* (OASIS Standard). https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html